Continuous Deployment

Automated, Repeatable Deploys

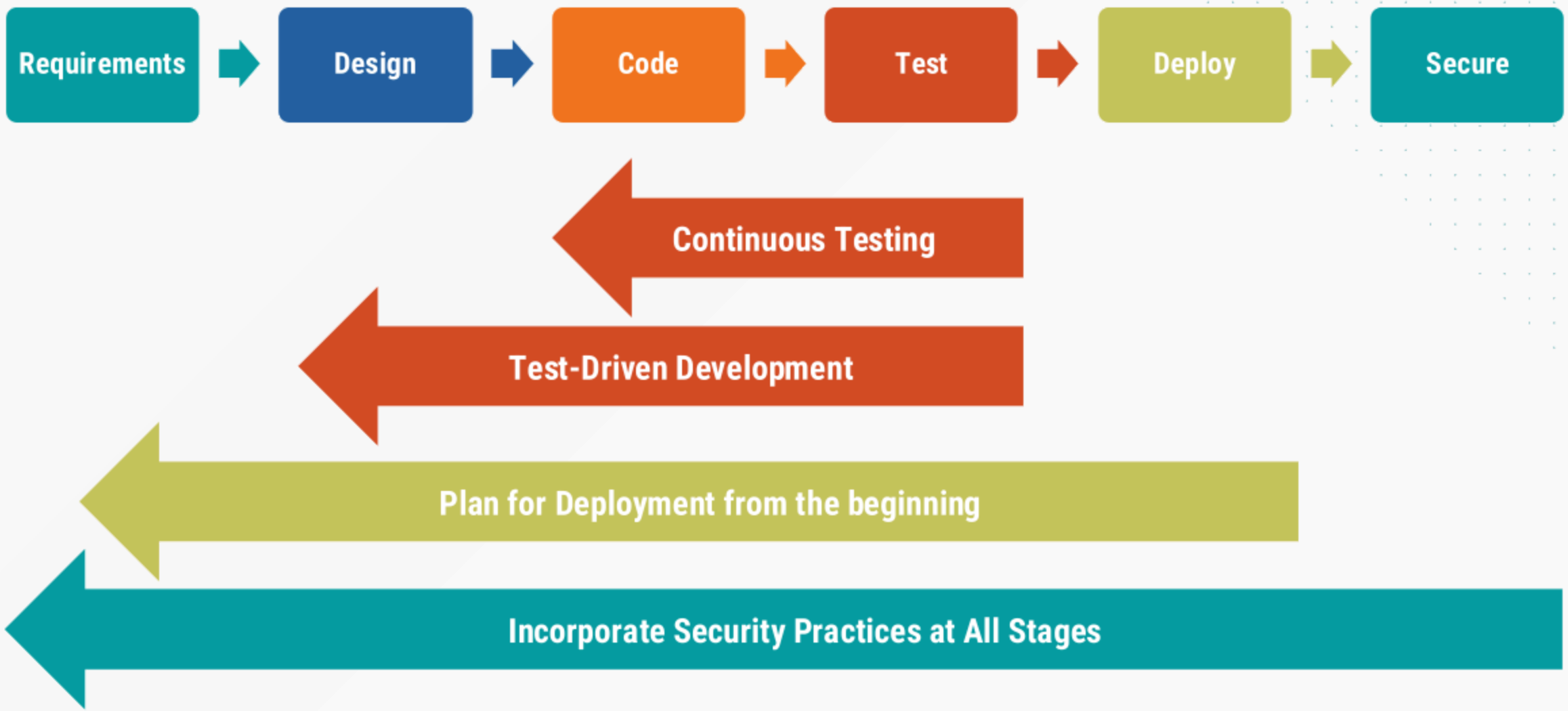
Our Project

1. Managed with git
2. Running Docker containers
3. Orchestrated with Kubernetes
4. Tested in Gitlab
5. **Deployed with ArgoCD**
6. Monitored with Prometheus and Grafana

DevSecOps: *Shift Left*



DevSecOps: *Shift Left*



Continuous Delivery and Deployment

Three related terms for continuous software development:

- **Continuous Integration (CI):** Developer changes are continuously integrated merged into *main* branch
- **Continuous Delivery (CD):** The *main* branch is always in a state ready to be deployed
- **Continuous Deployment (CD):** The *main* branch is automatically deployed when changes land

Continuous Delivery is Mostly Culture

For *main* to be deployable, all team members must cooperate

- Trunk-based development with small, frequent changes ensure changes reach main quickly, deltas between releases small
- Extensive testing gives confidence in frequent releases
- Failing tests on *main* is an emergency
- Backwards compatibility makes deploys easier; allows rollbacks

Continuous Deployment Requires Tooling

Same culture as for continuous delivery, but even more strict
→ No human in the loop to stop a bad deployment

Also need tooling to run deployments automatically

Recall first devsecops principle: **Repeatability and automation**

CD Continuum

The deployment automation necessary for continuous deployment is useful even when deployments are not triggered automatically

- Automated processes are repeatable, ensuring that deployments work every time
- Automated processes are easier, making a team practicing continuous delivery willing to deploy more often
- Builds confidence for eventual transition to continuous deployment

First Step: Deploying the Docker Image

- Already building the *web-service* docker image in CI
- If tests pass, we should publish it
- Docker images often stored in a **container registry**

Container Registries

We have already been pulling images from several public registries:

- Docker Hub for the *ollama* container
- GitHub Container Registry for the *uv* container

GitLab provides a container registry for each project
→ See *Deploy* ⟩ *Container registry*
→ Available at **registry.gitlab.com/*username/project***

Deploying a Docker Image

Docker images are deployed by tagging with the full registry

```
# Tag at build time
docker build ... -t registry.gitlab.com/user/project/image:tag

# Or add a tag to an existing image
docker tag local:latest registry.gitlab.com/user/project/image:tag
```

Then push the the image to the registry:

```
docker push registry.gitlab.com/user/project/image:tag
```

Authentication and Container Registries

Authenticate with a container registry with the *login* command:

```
docker login registry.gitlab.com
```

Authentication is necessary to push to registry, may be necessary to pull from registry

GitLab container registries share project permissions, by default

See *Settings* ❭ *General* ❭ *Visibility, project features, permissions*

When to Push an Image

We have decided to push images:

- Only when tests are passing
- Only from the *main* branch
  - These are the ones that could be deployed
- Only when the image has actually changed
  - Identifying this can be tricky

Identifying Images

- Want a way to identify images from their contents
  - We know when the image has changed
  - Can use this to tag the image

1. **Use the git sha.** Picks up all changes to repository.
2. **Use docker image digest.** Not reproducible with default settings.
3. **Derive from image directory contents.** Works if and only if rest of build process is reproducible.

Deriving an ID from Directory Contents

The `git rev-parse` command generates a hash of file contents

```
git rev-parse HEAD:project/images/web-service
```

This will not be reproducible unless:

- The build process is reproducible
  - This is why we lock the versions of the packages we're using
- The base image is locked
  - It isn't now, but we will do that soon

Calculate Image Tag During Build

Update the *build-image* job in `.gitlab-ci.yml`:

```
build-image:
  # ...
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
    IMAGE_NAME: $CI_REGISTRY_IMAGE/web-server
  script:
    - IMAGE_TAG=$IMAGE_NAME:$(git rev-parse HEAD:project/images/web-service)
    - echo "Building image ${IMAGE_TAG}"
    - docker build project/images/web-service -t $IMAGE_TAG
    - docker save -o web-service.tar $IMAGE_TAG
  # ...
```

Calculate Image Tag During Build

```
build-image:
  # ...
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
    IMAGE_NAME: $CI_REGISTRY_IMAGE/web-server
  script:
    - IMAGE_TAG=$IMAGE_NAME:$(git rev-parse HEAD:project/images/web-service)
    - echo "Building image ${IMAGE_TAG}"
    - docker build project/images/web-service -t $IMAGE_TAG
    - docker save -o web-service.tar $IMAGE_TAG
  # ...
```

`$CI_REGISTRY_NAME` provided automatically by GitLab CI/CD

Calculate Image Tag During Build

```
build-image:
  # ...
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
    IMAGE_NAME: $CI_REGISTRY_IMAGE/web-server
  script:
    - IMAGE_TAG=$IMAGE_NAME:$(git rev-parse HEAD:project/images/web-service)
    - echo "Building image ${IMAGE_TAG}"
    - docker build project/images/web-service -t $IMAGE_TAG
    - docker save -o web-service.tar $IMAGE_TAG
  # ...
```

Calculate image tag from directory contents

Pass Image Tag to Later Job

Write file with environmental variables, to be loaded in later job

```
build-image:
  # ...
  script:
    - IMAGE_TAG=$IMAGE_NAME:$(git rev-parse HEAD:project/images/web-service)
    - echo "IMAGE_TAG=${IMAGE_TAG}" >> build.env
    - echo "Building image ${IMAGE_TAG}"
    # ...
  artifacts:
    # ...
    reports:
      dotenv: build.env
```

Add a Deploy Stage

```yaml
stages:
  - test
  - build
  - deploy
# ...
deploy-image:
  stage: deploy
  dependencies:
    - build-image
  image: docker:27-cli
  services:
    - docker:27-dind
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Add a new stage, *deploy* to run after the *build* stage

The *deploy-image* job is configured similarly to the *build-image* job

Add a Deploy Stage

```yaml
stages:
  - test
  - build
  - deploy
# ...
deploy-image:
  stage: deploy
  dependencies:
    - build-image
  image: docker:27-cli
  services:
    - docker:27-dind
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Marking this as a dependency of the *build-image* job ensures the `$IMAGE_TAG` variable defined there will be inserted here

Add a Deploy Stage

```yaml
stages:
  - test
  - build
  - deploy
# ...
deploy-image:
  stage: deploy
  dependencies:
    - build-image
  image: docker:27-cli
  services:
    - docker:27-dind
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Only run this job on the *main* branch

Push the Image to the Container Registry

```
deploy-image:
  # ...
  before_script:
    - |
      echo "$CI_REGISTRY_PASSWORD" |
      docker login $CI_REGISTRY -u $CI_REGISTRY_USER --password-stdin
  script:
    - docker load -i web-service.tar
    - docker push $IMAGE_TAG
```

Push the Image to the Container Registry

```
deploy-image:
  # ...
  before_script:
    - |
      echo "$CI_REGISTRY_PASSWORD" |
      docker login $CI_REGISTRY -u $CI_REGISTRY_USER --password-stdin
  script:
    - docker load -i web-service.tar
    - docker push $IMAGE_TAG
```

GitLab CI/CD provides ephemeral `$CI_REGISTRY_USER` and `$CI_REGISTRY_PASSWORD` for container registry access

Push the Image to the Container Registry

```
deploy-image:
  # ...
  before_script:
    - |
      echo "$CI_REGISTRY_PASSWORD" |
      docker login $CI_REGISTRY -u $CI_REGISTRY_USER --password-stdin
  script:
    - docker load -i web-service.tar
    - docker push $IMAGE_TAG
```

Load the image built in *build-image*. It's already tagged with the registry name, so we can push it.

Activity: Kick off a Pipeline Run

1. Commit the changes to `.gitlab-ci.yml` to the *main* branch
2. Push the changes to GitLab
3. Watch the Pipeline execute
    - It should have a *deploy* stage
    - This should push the docker image to the registry
4. Copy the image tag from the output (we'll need it soon)
5. Check your project's container registry for an image

Accessing Registries from within K8s

- Images in public registries can be referenced with their full path in the `image` value in K8s manifests
- To access images in a private registry:
  1. Create a *docker-registry* Secret:

```
kubectl create secret docker-registry mycreds --docker-server=mysrv \
  --docker-username=user --docker-password=pswd --docker-email=email
```

  2. In your pod spec, add `imagePullSecrets`:

```
  imagePullSecrets:
    - name: mycreds
```

Activity: Use Image from Registry

1. Edit `project/services/web-service.yaml`
   - Switch image from `web-container` to tag from CI output
   - Remove `imagePullPolicy: Never`
2. Deploy the new version of the *web-service* with `kubectl apply -f web-service.yaml`
3. The *web-service* pods should restart

Automatically Update Image Tag

- When we deploy a new image, we want it to be used automatically
- Could tag image as *:latest*, reference that in manifest, but:
  1. K8s does not check for updated images, by default
  2. We wouldn't be able to roll back to old images
- Instead, create a new git commit to update the image tag *within the CI pipeline*

Modify the *deploy-image* Job

Set up a (arbitrary) username and email for git

```
deploy-image:
  # ...
  before_script:
    - |
      echo "$CI_REGISTRY_PASSWORD" |
      docker login $CI_REGISTRY -u $CI_REGISTRY_USER --password-stdin
    - git config --global user.name "GitLab CI"
    - git config --global user.email "ci@gitlab.com"
```

Modify the *deploy-image* Script

```
deploy-image:
  # ...
  script:
    - |
      sed -i "s|image: ${CI_REGISTRY_IMAGE}.*|image: ${IMAGE_TAG}|g" \
        project/services/web-service.yaml
    - git diff --quiet || TAG_CHANGED=true
    - |
      if [ "$TAG_CHANGED" = true ]; then
        echo "Changes to docker image.  Pushing new image."
        docker load -i web-service.tar
        docker push $IMAGE_TAG
        git add project/services/web-service.yaml
```

—Continues→

Modify the *deploy-image* Script

```
deploy-image:
  # ...
  script:
    - |
      sed -i "s|image: ${CI_REGISTRY_IMAGE}.*|image: ${IMAGE_TAG}|g" \
        project/services/web-service.yaml
    - git diff --quiet || TAG_CHANGED=true
    - |
      if [ "$TAG_CHANGED" = true ]; then
        echo "Changes to docker image.  Pushing new image."
        docker load -i web-service.tar
        docker push $IMAGE_TAG
        git add project/services/web-service.yaml
```

—Continues→

Modify the *deploy-image* Script

```
deploy-image:
  # ...
  script:
    - |
      sed -i "s|image: ${CI_REGISTRY_IMAGE}.*|image: ${IMAGE_TAG}|g" \
        project/services/web-service.yaml
    - git diff --quiet || TAG_CHANGED=true
    - |
      if [ "$TAG_CHANGED" = true ]; then
        echo "Changes to docker image.  Pushing new image."
        docker load -i web-service.tar
        docker push $IMAGE_TAG
        git add project/services/web-service.yaml
```

—Continues→

Modify the *deploy-image* Script

←*Continued*—

```
      git commit -m "Update web-service image tag [skip ci]
    Now set to ${IMAGE_TAG}"
      git push https://...git HEAD:main
    else
      echo "Docker image unchanged."
    fi
```

The `[skip ci]` message tells GitLab CI to not trigger a pipeline for this commit

⇒ Avoid causing a loop

Modify the *deploy-image* Script

←Continued—

```
git commit -m "Update web-service image tag [skip ci]
      Now set to ${IMAGE_TAG}"
        git push https://...git HEAD:main
      else
        echo "Docker image unchanged."
      fi
```

Full URL for *push* command:

```
https://oauth2:${GITLAB_WRITE_TOKEN}@${CI_SERVER_HOST}/
${CI_PROJECT_PATH}.git
```

Creating an Access Token

The CI script will need a GitLab Access Token in order to push to repo

1. Go to *Settings* ⟩ *Access tokens* and click *Add new token*
2. Add a *Name* and *Description*
3. Set the role to be *Maintainer*
4. Enable the *write_repository* scope
5. Click *Create project access token*
6. Copy the access token string

Using the Access Token in the Pipeline

The access token must be made available in the `$GITLAB_WRITE_TOKEN` environmental variable

1. Go to *Settings* ❭ *CI/CD* ❭ *Variables*
2. Under *Project variables*, click *Add variable*
3. Set visibility to *Masked and hidden*
4. Set the *Key* to be `GITLAB_WRITE_TOKEN`
5. Set the *Value* to be the token copied above
6. Click *Add variable*

Test the Pipeline

Commit the changed `.gitlab-ci.yml` file and push to GitLab

The pipeline should kick off

You may or may not get a commit editing the image tag, depending on how you had formatted that line by hand

If a new commit was created, pull it to your local repo with `git pull gitlab main`

Activity: Switch Dockerfile to Use Digests

The `project/images/web-service/Dockerfile` uses tags for base images
→ A change in that tag could change the resultant image

1. Get the SHA digests for the base images
   - For local image: `docker images --digests image:tag`
   - For remote image: `docker manifest inspect image:tag`
2. Adjust the Dockerfile to use the digest instead of the tag
   `FROM image@sha256:012...def`
3. Commit and push. This will start a pipeline that builds a new image.
4. Pull the resultant commit: `git pull gitlab main`

Deploying Updated K8s Manifests

We need to automatically deploy manifests when they change in *main*

Two general approaches:

1. **Push-based:** CI pipeline includes a deploy step
2. **Pull-based / GitOps:** Agent watches for changes in repo, reconciles system to desired state

Pushed-based	GitOps
Pipeline triggers deployment process	Controller reconciles repo with cluster state
Procedural definition; may miss drift from manual changes	Declarative definition; reverts manual changes
CI needs edit access to cluster	Cluster needs read access to CI
Failure shows up in pipeline	Need monitoring to detect failure
Pipeline plus CLI tools	Special controller (*ArgoCD*, *Flux*)

ArgoCD

ArgoCD is a GitOps continuous deployment tool for Kubernetes

- Git repositories act as source of truth
- Works with YAML manifests, Helm charts, and other formats
- Runs as a Kubernetes **controller**
  - Adjusts state of cluster to match that required by manifests
- Configure with web UI, CLI, or declarative YAML
- Can run in production cluster (**standalone**) or on a separate mangement cluster (**hub-and-spoke**)

Standalone

- ✅ Simple to set up
- ✅ No external access to API server
- 🔴 Powerful permissions inside production cluster
- 🔴 Broken cluster cannot repair itself

Hub and Spoke

- ✅ Single resource can control all clusters
- ✅ Can repair broken clusters
- 🔴 Contains credentials for all clusters
- 🔴 Single point of failure

Point of Pedagogy

We will be installing ArgoCD in our local development environment

This is *not* standard practice—ArgoCD is used to deploy staging and production clusters

We are doing this only to have a place to play with ArgoCD—don't do this on real development environments

Installing ArgoCD

```
# Create namespace
kubectl create namespace argocd
# Create resources
kubectl apply -n argocd --server-side --force-conflicts \
-f https://raw.githubusercontent.com/\
argoproj/argo-cd/stable/manifests/install.yaml
```

This install.yaml contains a number of K8s manifests
→ This could be vendored in our git repository

`--server-side`: Avoid issues with large CRDs
`--force-conflicts`: Needed for upgrades

Activity: What Did ArgoCD Install?

Use `kubectl` to look for pods, deployments, and services in the *argocd* namespace

You may find `kubectl get all -n argocd` useful

ArgoCD Core

The default installation includes many features beyond the basic GitOps tools

ArgoCD Core provides just enough to conduct GitOps

Consider using this on production systems, to decrease target area

ArgoCD Web UI

The *argocd-server* service is a web UI to ArgoCD

```
kubectl port-forward -n argocd services/argocd-server 8080:443
```

This server is speaking HTTPS. To connect to it through the Codespaces proxy:

1. Go to *Ports* tab in the Codespaces bottom pane
2. Right click on port 8080
3. Click *Change Port Protocol* ⟩ *HTTPS*

Logging in to the Web UI

ArgoCD can support multiple users, each with different permissions

A default user named *admin* has been created

The password is stored in a K8s Secret. You can view it with the following:

```
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath='{.data.password}' | base64 -d
```

Log in with user *admin*

ArgoCD Applications

ArgoCD deploys **Applications**

We'll make an Application that deploys the *web-server* and *ollama* services

Click *+ New App* to create an Application

Creating an Application, pt. 1

- The *Application Name* needs to be a valid K8s identifier
  - We'll use `dso-application`
- *Project Name* should be `default`
  - User permissions can be allocated per project
- In *Source*, set the *Repository URL* to the HTTPS git from GitLab
  - This requires a public-visible project
  - ArgoCD can be set up to access private repositories
- Set *Path* to `project/services`
  - This is the path in our repo containing the manifests

Creating an Application, pt. 2

- In *Destination*, set *Cluster URL* to `https://kubernetes.default.svc`
  - This refers to the cluster itself, as we're using standalone mode
- Set *Namespace* to `dso-project`
- Click *Create*

The new Application should pick up all existing components defined in `project/services`, but is unlikely to be in sync
→ State of cluster does not match specification in git
⇒ Press the *Sync* button to synchronize the deployment with git

App Health

Ideally, the Application will be reported as *Healthy*

If the health is stuck in *Progressing*, it may be due to a service trying to use a LoadBalancer
→ Minikube doesn't provide real load balancers, and Argo CD notices
⇒ Switch service type to *ClusterIP*

Activity: Deploy a Change

1. Change the number of replicas in the *web-service* Deployment
2. Push the changes to GitLab master
3. Within 3 minutes, ArgoCD should notice the change and report that the Application is not in sync
4. Synchronize the Application
5. Check that the number of pods for the *web-service* has changed

Configuration Mechanisms

How does the use of the Web UI align with DevSecOps principles?

1. Repeatability and automation
2. Configuration as code
3. Store configuration in version control
4. Declarative, not procedural
5. Livestock, not pets
6. Testing

Configuration Mechanisms

How does the use of the Web UI align with DevSecOps principles?

1. ~~Repeatability and automation~~
2. ~~Configuration as code~~
3. ~~Store configuration in version control~~
4. ~~Declarative, not procedural~~
5. Livestock, not pets
6. Testing

Declarative Configuration

Applications should be defined declaratively, in configuration files

Applications are tracked by Kubernetes:

```
kubectl get applications -n argocd
```

They are a **Custom Resource Definition (CRD)**

- Mechanism to define custom *kinds* in Kubernetes
- A custom **controller** can use CRDs to understand what to deploy

Extract Custom Resource Configuration

```
# Create a new folder
mkdir project/argocd
# Write Custom Resource manifest to disk
kubectl get applications -n argocd dso-application -o yaml > \
  project/argocd/apps.yaml
```

This file will contain state information that's unneeded in manifest

- Delete the entire `service` structure
- Delete all entries in `metadata`, except `name` and `namespace`

Resultant file should look like...

Application Configuration

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: dso-application
  namespace: argocd
spec:
  destination:
    namespace: dso-project
    server: https://kubernetes.default.svc
  project: default
  source:
    path: project/services
    repoURL: https://gitlab.com/pragdata/devsecops.git
    targetRevision: HEAD
```

Edit Applications with `kubectl`

Now, applications can be created and edited with `kubectl apply`:

```
kubectl apply -f project/arcocd/apps.yaml
```

This will likley give a warning about a missing *last-applied-configuration* annotation
→ This should only happen the first time, and only because we created this Application through the Web UI first

ArgoCD can Synchronize Automatically

```
spec:
  # ...
  syncPolicy:
    automated:
      enabled: true
```

Additional options:

`prune: true`: Delete resources no longer defined in git

`allowEmpty: true`: Allow application with empty resoruces

`selfHeal: true`: Overwrite manual changes to cluster state

Activity: Trigger an Automated Sync

1. Apply the Application with automated synchronization
2. Change the number of replicas in the *web-service* Deployment
3. Push this change to GitLab
4. In several minutes, ArgoCD should notice the change, synchronize the cluster, and change the number of *web-service* pods

Namespace Confusion

Currently specifying namespace in Argocd and resource manifests
→ Likely confusion if they were to differ

Best practice: Specify namespace only in ArgoCD

ArgoCD Manages Namespaces

1. Remove *dso-project* namespace definition from `web-service.yaml`
2. Remove namespace from metadata for all resources in `web-service.yaml` and `ollama.yaml`
3. Let ArgoCD create namespace if needed

```yaml
spec:
  # ...
  syncPolicy:
    # ...
    syncOptions:
      - CreateNamespace=true
```

A Pedagogical Annoyance

- Once these changes are deployed, ArgoCD won't fully synchronize
  - ArgoCD sees that namespace no longer defined in git
  - Namespace still exists (unless pruning turned on)
- Trigger a manual sync, enabling pruning
  - The entire *dso-project* namespace is deleted
  - We are not properly synced
- Manually synchronizing again should fix everything

This is all because we are converting the existing resources, instead of starting from scratch

Helm Applications

ArgoCD can deploy Kubernetes resources defined as

- Kubernetes manifests
- Helm charts
- Kustomize applications
- OCI images

For a Helm chart, specify location of chart and values file

Create an Application for Garage

Add a new YAML document to `project/argocd/apps.yaml`

```yaml
---
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: garage
  namespace: argocd
spec:
  project: default
```

Create an Application for Garage

```yaml
spec:
  # ...
  sources:
    - repoURL: https://datahub-local.github.io/garage-helm
      chart: garage
      targetRevision: 0.8.0
      helm:
        valueFiles:
          - $values/project/helm/garage.yaml
    - repoURL: https://gitlab.com/pragdata/devsecops.git
      targetRevision: HEAD
      ref: values
```

Create an Application for Garage

```yaml
spec:
  # ...
  sources:
    - repoURL: https://datahub-local.github.io/garage-helm
      chart: garage
      targetRevision: 0.8.0
      helm:
        valueFiles:
          - $values/project/helm/garage.yaml
    - repoURL: https://gitlab.com/pragdata/devsecops.git
      targetRevision: HEAD
      ref: values
```

Multiple sources for this Application

Create an Application for Garage

```yaml
spec:
  # ...
  sources:
    - repoURL: https://datahub-local.github.io/garage-helm
      chart: garage
      targetRevision: 0.8.0
      helm:
        valueFiles:
          - $values/project/helm/garage.yaml
    - repoURL: https://gitlab.com/pragdata/devsecops.git
      targetRevision: HEAD
      ref: values
```

The chart is defined in the Helm repository

Create an Application for Garage

```yaml
spec:
  # ...
  sources:
    - repoURL: https://datahub-local.github.io/garage-helm
      chart: garage
      targetRevision: 0.8.0
      helm:
        valueFiles:
          - $values/project/helm/garage.yaml
    - repoURL: https://gitlab.com/pragdata/devsecops.git
      targetRevision: HEAD
      ref: values
```

The *values* file is stored in our git repository

Create an Application for Garage

```
spec:
  # ...
  sources:
    - repoURL: https://datahub-local.github.io/garage-helm
      chart: garage
      targetRevision: 0.8.0
      helm:
        valueFiles:
          - $values/project/helm/garage.yaml
    - repoURL: https://gitlab.com/pragdata/devsecops.git
      targetRevision: HEAD
      ref: values
```

Update the URL to match your GitLab repository

Create an Application for Garage

```
spec:
  # ...
  sources:
    - repoURL: https://datahub-local.github.io/garage-helm
      chart: garage
      targetRevision: 0.8.0
      helm:
        valueFiles:
          - $values/project/helm/garage.yaml
    - repoURL: https://gitlab.com/pragdata/devsecops.git
      targetRevision: HEAD
      ref: values
```

A reference to the *values* repository in the *valueFiles* for the chart

Create an Application for Garage

```
spec:
  # ...
  destination:
    namespace: garage
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      enabled: true
    syncOptions:
      - CreateNamespace=true
```

Apply this updated manifest to create a new Application

Deploying Only After Tests Pass

With pushed-based CD, deployment can be gated on tests passing

With GitOps CD, need tests to pass before changes merged

- Be careful and only merge to main when tests pass on branch
- Only write change to manifest (eg new image) once tests pass
- Have a deployment repo, push changes there once tests pass
- Have a deployment branch, merge there once tests pass

Rollbacks

ArgoCD tracks the last 10 synchronizations for each application

Can rollback to a previous state with the Web UI or CLI

Beware: If automatic synchronization enabled, ArgoCD will notice discrepancy and re-sync to git HEAD

GitOps Solution: Revert the problematic commit and deploy again
$\Rightarrow$ Keep all actions in git history

Rollouts

- By default, ArgoCD uses native K8s rolling updates
  - Pods are gradually restarted with the new configuration
  - Ensures level of service throughout rollout
- More rollout strategies provided by Argo Rollouts
  - **Blue-green deployments:** Run two instances in parallel
    - Test new deployment before switching production traffic
    - Instant rollback to previous deployment
  - **Canary deployments:** Gradually shift traffic to new deployment
    - Monitor health metrics before taking next step

Activity: Stand Up New Cluster

1. Push all changes to GitHub
2. Start a new Codespace
3. Start minikube
4. Install ArgoCD
5. Install our applications

Future Activity: Capstone Project

In the coming Capstone session, we'll be working through a scenario using these tools.

Some of you will need to drive configuration changes through review, merge, and deployment. We will need your GitLab usernames to add you to the project.

It may prove useful for you to be able to share your screen.